

Expected Result	Goal	Test Procedure	ID	Module
count reaches 9 and carry_out rises to '1'.	Frequency division test	Inject 9 clock cycles (clk)	1.1	counter
count resets to 4'd0 and carry_out drops to '0'.	Reset mechanism test	Assert rst (pull to '0').	1.2	
`state` increments from 1 up to 6 sequentially, and `out` updates to match each RPM countdown goal.	Sequential speed increment test	Apply 5 consecutive pulses to `speed_sel` starting from reset state.	2.1	speed_control_fsm
At S6, the next pulse drops `state` to S5 (counting down). At S1, the next pulse raises `state` to S2 (counting up).	Ping-pong turnaround logic at boundaries	At max speed (S6), apply another pulse to `speed_sel`. Then navigate down to S1 and pulse again.	2.2	
`state` instantly forces back to 4'd1 (S1) and `out` resets to 28'd750000.	Asynchronous reset verification	Navigate to S4, then assert `resetb` = 1'b0`.	2.3	
During the state traversal test, compare out against the expected divider value for every current_state.	Verify output values per state	Each state produces the correct out value: S1=750000, S2=375000, S3=250000, S4=187500, S5=150000, S6=125000.	2.4	
Start from S1. Set speed_sel=1 and hold it high for several clock cycles. Then release to 0 and press again.	Verify long press is counted as one press	Holding speed_sel high for multiple clock cycles creates only one speed_pulse and causes only one transition, for example S1 -> S2. No additional transitions occur until speed_sel is released and pressed again.	2.5	
seg_out displays the correct binary pattern according to the case statement.	7-segment decoder accuracy	Apply values 4'b0000 through 4'b1001 to current_digit.	3.1	seg_7
seg_out outputs 4'b0 as defined in the default case.	Default case handling	Apply an undefined value > 9 (e.g., 4'b1010 to 4'b1111) to current_digit.	3.2	
`quarter_active` rises to '1' on the first press. The second press is safely ignored (system remains locked in current run).	Quarter-turn activation and lockout mechanism	Pulse `quarter` button while `on` = 1'b0`. Cancel `quarter_active` is '1', pulse `quarter` again.	4.1	mode
quarter_active remains '1'. driver_enable continues to pass freq_out seamlessly until exactly 100 steps are completed, then quarter_active drops to '0'.	Quarter-turn persistence during mode collision	Assert continuous mode (on = 1'b1) while a quarter-turn is active, then de-assert on = 1'b0 before 100 steps are reached.	4.2	
`quarter_active` and `quarter_count_en` drop to '0' on the next clock edge, cleanly stopping the single-turn execution.	Quarter-turn completion sequence	Assert `quarter_done` = 1'b1` while `quarter_active` is '1'.	4.3	
`quarter_active` and `previous_q_btn_mode` drop immediately to '0', effectively killing the quarter-turn command.	Asynchronous reset verification	Force `resetb` = 1'b0` while `quarter_active` is currently '1' (mid-turn).	4.4	

quarter_active remains '1'. Motor runs seamlessly. Once 100 steps are reached, quarter_active drops to '0', but driver_enable continues to pass freq_out seamlessly because on is still '1'.	Continuous mode takes over smoothly	Assert on = 1'b1 while a quarter-turn is active. Keep on = 1'b1 held high well beyond the 100-step completion of the quarter-turn.	4.5	driver_control
`pulses_out` transitions sequentially through all 8 states (`st_0` to `st_7`) on every single active `enb` pulse.	Half-step sequence progression	Set `step_size` = 1'b0, `direction` = 1'b0, and provide continuous `enb` pulses.	5.1	
`skip_toggle` alternates state transitions every 2 `enb` cycles, skipping intermediate states (`st_1->st_3->st_5->st_7`) to maintain constant mechanical RPM.	Full-step pulse skipping and timing	Set `step_size` = 1'b1 (Full step) and provide continuous `enb` pulses.	5.2	
The FSM state sequence immediately reverses its tracking order (e.g., transitions from `st_3` back to `st_2` then `st_1`).	Real-time direction reversal	Flip the `direction` switch from '1' to '0' mid-operation.	5.3	
`current_state` reverts immediately to `st_1` and `skip_toggle` clears to '0', resetting the motor sequence.	Asynchronous reset verification	Force `resetb` = 1'b0 while `current_state` is at `st_4` and `skip_toggle` is '1'.	5.4	
Motor coils pulse exactly 100 times at the speed rate dictated by `HEX1`, then the counter fires `carry_out` and cleanly halts rotation.	Full system integration: Quarter-turn execution	Pulse `quarter` with `on` = 1'b0. Observe `quarter_counter` value and `pulses_out`.	6.1	step_motor_top
Coil switching frequency updates dynamically with `HEX1`, rotation direction flips instantly, and velocity stays uniform across step-size switches.	Full system integration: Live parameter changes	Set `on` = 1'b1. Dynamically cycle `speed_sel`, flip `direction`, and toggle `step_size`.	6.2	
Continuous mode takes priority (`on` overrides `quarter`), the motor runs continuously, and `quarter_active` remains '0'.	Simultaneous input collision	Assert `quarter` button and `on` switch to '1' at the exact same clock edge.	6.3	
The motor immediately changes its rotation speed, but continues to count cleanly up to exactly 100 steps without losing count.	Parameter change mid-operation	While `quarter_active` is running (e.g., at step 50), dynamically change `speed_sel`.	6.4	
Asynchronous reset takes absolute priority. System halts immediately, all counters drop to 0, and FSMs return to initial states safely.	Reset priority during state transition	Pull `resetb` to '0' precisely when `quarter_carry_out` is generated (at step 100).	6.5	
All internal counters instantly clear to 0, `state` returns to S1, and `pulses_out` and `sev_seg` displays immediately blank out to their default reset values.	Global mid-operation reset verification	Set `on` = 1'b1, let the motor spin continuously at S4, then suddenly assert `resetb` = 1'b0 mid-operation.	6.6	